Run report: auxiliary loss, n = 72, edge-bias attention, depth

Analyzing the Effect of Network Width on Graph Transformers · 4–8 November 2026

1. Summary in plain words

The thesis asks how **wide** a graph transformer must be to solve a graph problem, and how that width grows as the graphs get bigger. Section 2 explains every term used below.

1. **A training-only hint slows the growth.** Asking the middle layers to also predict 2-, 4- and 8-step reachability during training (and dropping that extra output at test time) makes the required width grow like $n^{1.30}$ instead of $n^{1.74}$.
2. **At n = 72 the plain model ran out of data, not width.** It learned the training graphs at the width the trend predicts (about 81) but did not generalise to new graphs; doubling the training set mostly fixed that.
3. **Putting the edges into the attention makes the problem almost free.** When each layer is told directly "look at your neighbours", 4–6 numbers per node are enough for every graph size from 32 to 56 nodes, where the plain model needs 22–60. The width no longer grows with n in that range.
4. **Past about 96 nodes those models get stuck.** Training stalls at 85–93 % of pairs right, and whether a run gets unstuck looks random.
5. **The cause is depth, not width.** Letting a stuck model run its 6 layers twice (no new parameters) unsticks every one of them; a 12-layer model trained from scratch never gets stuck. Changing the attention bias or the learning-rate schedule barely helps. The same holds at 128 nodes, and using more, narrower heads does not remove the stall either.
6. **With 12 layers the needed width falls back down.** About 7 numbers per node are enough at 72 and 96 nodes, close to the 4–6 needed at 32–56 nodes. At 128 nodes it is about 11–15 with 12 or 16 layers (against 23 to more than 32 with 6), so 16 layers do not bring it back to 7. Single runs are erratic: a run can pass at one width and fail at the next, and deeper models sometimes collapse late in training. So for this encoding the width has to be measured with a depth that grows with the graph size, over more seeds.

The numbers behind these statements. Auxiliary loss: m^* exponent 1.74 → 1.30 over n = 32–56. $n = 72$: train-fit $m^* = 81$; test never reaches 0.95 with 128k graphs, 256k graphs bring the train–test gap from 0.04–0.08 to 0.02. Edge bias on adjacency rows: $m^* = 5.6$, 5.7, 4.1, 5.9 at $n = 32$ –56 (head dimension 2), 13–15 at $n = 72$, 16–28 at $n = 96$, above 22 at $n = 128$ (depth 6); at depth 12 with head dimension 2, 7.0 / 6.9 at $n = 72$ and 4.8 / 7.3 at $n = 96$; at $n = 128$ (head dimension 8, 4 seeds) about 14.5 at depth 12 and 11 at depth 16, with one seed passing at $m = 8$ at each depth. Plateau study, seeds passing 0.95 out of 4: at $n = 96$, $m = 16$ depth 6 1, depth 12 4, layers run twice 3; at $n = 128$, $m = 24$ depth 6 0, depth 12 3, layers run twice 4.

2. Background: what is being measured

2.1. The task

A graph is a set of n nodes joined by edges. Two nodes are **connected** if you can walk from one to the other along edges, however many steps it takes. The model gets a graph and must answer, for every pair of nodes, "connected or not?": an $n \times n$ table of yes / no answers (the **reachability matrix**).

Example with 6 nodes in two separate pieces (**components**):

```
0 – 1 – 2      3 – 4 – 5
```

Nodes 0 and 2 are connected (two steps); 0 and 4 are not. The right answer table has "yes" for every pair inside {0, 1, 2} and inside {3, 4, 5}, and "no" for every pair across.

The graphs used here (`swap` graphs) are built so that no shortcut works: every graph is two blobs, and the only difference between "one component" and "two components" is how two edges are attached far apart. Every node's neighbourhood looks the same in both cases, so the model has to follow paths across the whole graph to answer.

2.2. What the model sees

The model does not see a picture. Each node becomes one **token**, a row of numbers, and the model reads all n tokens at once. In the basic encoding (**adjacency rows**) node i's token has a 1 for itself and every node it touches directly, 0 elsewhere. For the example:

token of node	0	1	2	3	4	5
0	1	1	0	0	0	0
1	1	1	1	0	0	0
2	0	1	1	0	0	0
3	0	0	0	1	1	0
4	0	0	0	1	1	1
5	0	0	0	0	1	1

So each node starts out knowing only its direct neighbours. To learn that 0 and 2 are connected the model must combine node 0´s row with node 1´s row, and for longer paths it must repeat this many times.

2.3. Width, depth and heads

The model is a stack of **layers** (the **depth**, 6 unless stated). In each layer every node looks at the other nodes (**attention**) and updates its own vector of numbers. The length of that vector is the **width** m: how much each node can remember about the graph. At the end the model compares the final vectors of every two nodes and outputs yes or no.

Each layer's attention is split into several **heads**, each a separate way of looking at the other nodes. A head uses a fixed number of the m numbers (the **head dimension**, 8 unless stated), so a model has $\frac{m}{8}$ heads; $m = 8$ means a single head. Using head dimension 2 allows widths below 8 with more than one head.

2.4. How the results are read

- **Pair accuracy:** the fraction of the $n \times n$ answers that are right, on graphs the model never saw in training (**test**). Since most pairs in these graphs are connected, answering "yes" to everything already scores about 0.75 (**always-yes**); a model stuck there has learned nothing.
- **Critical width m^*:** the smallest width whose test pair accuracy reaches 0.95, read off between the widths that were tried. Small m^* means the problem is easy for the model.
- **Exponent:** how m^* grows with n. "$m^* \propto n^{1.74}$" means doubling the graph size multiplies the needed width by $2^{1.74} \approx 3.3$; an exponent of 0 means the needed width does not grow at all.
- **Seeds:** each setting is trained 2–4 times from different random starts. When seeds disagree a lot, a single number is not trustworthy.
- **Train vs test:** if the model is right on its training graphs but not on new ones, it needs more data. If it is equally wrong on both, it could not learn the task at all (too narrow, too shallow, or stuck in training).

2.5. The encodings compared

Name	**What changes, in plain words**
adjacency rows (baseline)	Each node's token lists its direct neighbours.
+ 2-hop rows	The token also lists every node within two steps, computed before the model starts. The model gets two steps of path-following for free.

Name	What changes, in plain words
+ auxiliary loss	Input unchanged. During training only, layers 1–3 must also predict which nodes are within 2, 4 and 8 steps: a hint about *how* to solve the task.
edge bias (adj tokens)	Attention is pushed towards neighbours: every layer gets a bonus b on the attention score between nodes joined by an edge. Each layer then mostly passes information one step along the edges.
edge bias (random IDs)	Same bonus, but the token is just a random name for the node with no edges in it. Everything about the graph comes through the attention bonus.

The training recipe (**q3lr protocol**) is fixed for all of them: 128 000 training graphs, 24 000 training steps of 128 graphs, learning rate 10^{-3} decayed to 0, depth 6.

3. Runs in this report

Task: audited `swap` graphs, predict all-pairs reachability. Unless stated otherwise every run follows the q3lr protocol: 128 000 training graphs, 24 000 steps, batch 128, LR 10^{-3}, depth 6, head dimension 8, bilinear all-pairs read-out. The baseline input is the adjacency row $(A + I)_i$ per node token. The always-yes accuracy (predict every pair connected) is 0.745 at $n = 32$ and 0.764 at $n = 72$.

Sweep	Platform	Runs	Commit
`aux_dbl` (n = 32, 48)	Kaggle, 2×T4	16	`ba6f69e`
`q3n72`, `q3n72steps`	Colab T4	8 (3 + 5 after a lost session)	`8545788`
`ebp_*` pilot round 2	Kaggle, 2×T4	16	`3992a74`
`q3n72data`, `eb_rid72`	Colab T4, high RAM	8	`018ddb2`
`eb_adj`, `eb_rid`	Kaggle, 2×T4	64	`7b73b36`
`eb_adj_hd2`, `eb_rid_hd2`	Kaggle, 2×T4	64	`5778050`
`eb_adj_big` ($n = 72$–128)	Colab T4, high RAM, 3 sessions	26	`5778050`, `eef41f1`, `1d1e440`
`eb_surgery`, `ebx96_*`	Colab T4, high RAM	20 + 32	`af20a01`
`ebx128_*`, `eb_adj_hd2_72` / `_96`	Kaggle, 2×T4	32 + 16	`c9566a5`
`ebd12_*` (depth 12, $n =$ 72–128)	Colab T4, high RAM, 2 sessions	24	`176c6c5`, `9b4364f`
`ebd12hd2_*` (depth 12, head dim. 2)	Colab T4, high RAM	12	`fd2d069`
`ebd16_128`, `ebd12_128x`	Kaggle, 2×T4	12 + 6	`7c49f5c`

4. Auxiliary k-hop loss

In plain words: the model gets the same input, but during training its early layers are also graded on intermediate steps ("which nodes are within 2 steps? within 4?"). The question is whether this hint lets a narrower model learn the task.

Input and test-time model are the baseline. During training, blocks 1, 2 and 3 each feed a bilinear head that predicts 2-, 4- and 8-hop reachability (BCE, weight 0.3 per head). The heads are dropped at test time. Seeds 0 and 1; the baseline column uses the same two seeds.

n	adjacency rows	+ aux loss	saving
32	21.9 (20.7 / 23.1)	21.5 (20.2 / 22.7)	2 %
40	30.1 (30.8 / 29.4)	26.8 (25.8 / 27.8)	11 %
48	42.4 (41.2 / 43.6)	33.5 (32.0 / 35.0)	21 %
56	58.0 (59.2 / 56.7)	45.1 (44.2 / 46.0)	22 %
slope	1.74 (1.59–1.90)	1.30 (1.13–1.45)	

Table 1: m^*, mean (seed 0 / seed 1). Slope ranges: min–max over all one-seed-per-n combinations. Train-fit thresholds give 1.63 → 1.20.

The saving grows with n, so the loss changes the exponent rather than the constant. The two slope ranges do not overlap. The 2-hop input encoding reaches a similar exponent ($n^{1.27}$) at a far lower level (Figure 1): extra input information mainly lowers the level, while guiding the computation order mainly changes the growth. The aux heads are learned only in part: at the widest models the 4-hop head stays near 0.90 pair accuracy even where the main task reaches 0.995.

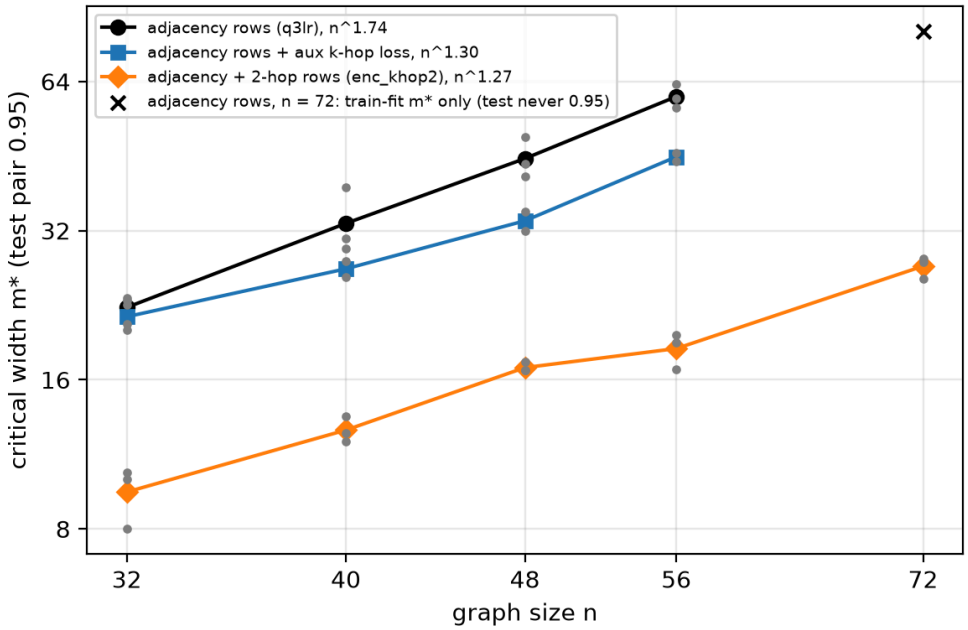


Figure 1: m^* against n, log–log. Grey dots: per-seed crossings. The cross at $n = 72$ is the train-fit m^*; the test threshold is not reached there.

5. Larger graphs: n = 72

In plain words: the trend from $n = 32$–56 predicts the plain model needs width about 81 at $n = 72$. We trained around that width to check the prediction.

5.1. 128k training graphs

Adjacency rows, q3lr protocol, widths 64 / 96 / 128, seeds 0 and 1; one width also at 48 000 steps. Final pair accuracy, test (train):

m	24k steps, seed 0 / 1	48k steps, seed 0 / 1
64	0.867 (0.916) / 0.898 (0.925)	–
96	0.947 (0.970) / 0.912 (0.973)	0.960 (0.990) / 0.914 (0.991)
128	0.941 (0.978) / 0.936 (0.986)	–

- Train-fit $m^* = 80.9$ (82.7 / 79.1); the $n^{1.63}$ line from $n = 32$–56 predicts 82.6. The five-size train-fit slope is 1.65.

- Test m^* is undefined: no run reaches 0.95 at 24k steps, and $m = 128$ is no better than $m = 96$. At 48k steps train reaches 0.99 but test moves to only 0.960 / 0.914.
- Train − test gap 0.04–0.08, against $\lesssim 0.02$ at $n = 56$.
- All runs stay at always-yes (0.764) for the first 8 of 24 epochs.

5.2. 256k training graphs

`q3n72data`: $m = 96$, 256 000 graphs, still 24 000 steps (12 epochs instead of 24).

training graphs	seed 0	seed 1
128k	0.947 (0.970)	0.912 (0.973)
256k	0.956 (0.974)	0.933 (0.955)

Test accuracy rises by 0.01–0.02 and the gap falls to 0.02, the level seen at $n \leq 56$. Seed 0 passes 0.95. Seed 1 does not, and its training accuracy is only 0.955 because it sees each graph half as often. A test m^* at $n = 72$ therefore needs 256k graphs, more steps, and widths around 96–192. Running $n = 96$ (`q3n96`) with 128k graphs is not useful.

6. Edges as attention bias

In plain words: normally each node must *learn* which other nodes are its neighbours before it can pass information along edges, and that takes width. Here we tell the attention directly: "pay extra attention to your neighbours". Each layer then acts like one step of spreading information along the edges, and after a few layers every node holds a summary of its whole component. Two nodes are connected exactly when their summaries match, which takes few numbers to store.

`EdgeBiasModel` adds $b_{\ell,h} \cdot (A + I)_{ij}$ to the attention score of every layer ℓ and head h; b is one learned scalar per layer and head, initialised to a chosen start value. Only direct edges are given, no distances. Two token types: *adj*, the usual adjacency row (n wide); *rid*, a fixed random $\pm 1/8$ vector of width 64 per node position (independent of n; the input then contains no edges at all).

6.1. Pilot, n = 32

Budget: first 16 000 graphs of the cached seed-0 set, 3000 steps (12 000 where marked), one seed. Round 1 ran on Colab (2026-11-03), round 2 on Kaggle. Final test pair accuracy:

tokens, bias start	steps	$m = 8$	16	32	64
adj, no bias (baseline)	3000	–	0.747	0.764	–
adj, 0	3000	0.828	0.865	0.931	–
adj, 2	3000	0.871	**0.999**	0.997	–
rid, 0 (round 1)	3000	–	0.745	0.745	0.745
rid, 0	12 000	–	–	0.935	0.947
rid, 2	3000	–	0.962	0.989	0.991
rid, 2	12 000	–	–	0.991	0.985
rid, 4	3000	–	**0.998**	0.988	0.993

- Round 1 (adj, start 0) reproduced exactly in round 2 (0.828 / 0.865 / 0.931).
- The zero start is what stopped random IDs: with 4× steps they leave always-yes, and with a start of 2 or 4 they learn within the first few hundred steps.
- With a start of 2–4, both token types reach ≥ 0.99 at $m = 16$ on 1/8 of the q3lr data and steps; the q3lr baseline needs $m^* = 22.4$ at $n = 32$ with the full budget.
- With a strong edge bias, attention mostly follows edges, so the model behaves partly like a message-passing network. This should be stated when the result is compared with the encodings.

6.2. Random IDs at n = 72

`eb_rid72`: rid tokens, bias start 4, full q3lr protocol (128k graphs), seeds 0 and 1. Final pair accuracy, test (train):

m	seed 0	seed 1
12	0.793 (0.787)	0.846 (0.846)
16	0.910 (0.912)	0.945 (0.943)
24	0.764 (0.764)	0.919 (0.932)

No run reaches 0.95, and train equals test, so the limit is optimisation, not data. The $m = 24$ seed-0 run reached 0.861 and then collapsed to always-yes from about epoch 9 until the end. A start of 4 combined with LR 10^{-3} may be too aggressive at this size. The full sweep shows the same problem at $m = 24$ for $n \geq 48$.

6.3. Full sweep, n = 32–56

`eb_adj` (adj tokens, start 2) and `eb_rid` (rid tokens, start 4), q3lr protocol, $m = 8, 12, 16, 24$, seeds 0 and 1. $m = 8$ is a single head. Final test pair accuracy, seed 0 / seed 1:

n	$m = 8$	12	16	24
eb_adj				
32	.996 / .992	1.00 / .996	1.00 / 1.00	1.00 / .999
40	.984 / .990	.993 / .992	.977 / .998	.997 / 1.00
48	.961 / .983	.979 / .987	.995 / .998	.994 / .998
56	.974 / .867	.956 / .975	.996 / .986	.998 / .994
eb_rid				
32	.990 / .995	.994 / .992	.995 / .995	.995 / .996
40	.964 / .972	.964 / .978	.986 / .984	.998 / .989
48	.928 / .882	.936 / .938	.956 / .993	.929 / .991
56	.814 / .905	.899 / .865	.971 / .985	.957 / .946

n	adjacency rows	eb_adj	eb_rid
32	22.4	≤ 8 / ≤ 8	≤ 8 / ≤ 8
40	33.1	≤ 8 / ≤ 8	≤ 8 / ≤ 8
48	44.7	≤ 8 / ≤ 8	14.7 / 12.8
56	59.6	≤ 8 / 10.9	14.7 / 14.7
72	train fit 81; test none	–	> 24 / > 24

Table 2: Critical width m^* (test pair ≥ 0.95), seed 0 / seed 1. "≤ 8": already passed at the smallest width tried. Adjacency rows: 3-seed mean from q3lr.

- Edge bias on adjacency rows removes almost all of the width cost: m^* stays at the grid floor up to $n = 48$, where plain adjacency rows need 45. No exponent can be fitted while most crossings sit on the floor; widths below 8 need a head dimension below 8.
- Random IDs carry no edges in the input and have a 64-wide read-in for every n. They match adjacency tokens up to $n = 40$ and need about 14 at $n = 48$ / 56, still a quarter of the plain adjacency width.
- Wider is not reliably better for random IDs: $m = 24$ scores below $m = 16$ at $n = 48$ (seed 0) and at $n = 56$ (both seeds). Train and test accuracy agree in every run, so this is an optimisation failure, like the collapse at $n = 72$ above, not overfitting.

6.4. Below the grid floor: head dimension 2

In plain words: the edge-bias models already passed at the smallest width tried (8), so the real m^* was unknown. Using 2 numbers per head allows widths 2–8.

`eb_adj_hd2` / `eb_rid_hd2`: as above, but head dimension 2, so $m = 2, 4, 6, 8$ has 1–4 heads. Final test pair accuracy, seed 0 / seed 1:

n	adj $m = 2$	4	6	8	rid $m = 4$	6	8
32	.745 / .746	.745 / .746	.991 / 1.00	1.00 / 1.00	.858 / .994	.995 / .998	.993 / .992
40	.750 / .750	.750 / .750	.966 / .993	.994 / .995	.859 / .838	.986 / .865	.931 / .975
48	.755 / .755	.977 / .919	.985 / .985	.988 / .991	.824 / .767	.950 / .935	.874 / .947
56	.759 / .759	.941 / .759	.979 / .884	.971 / .975	.804 / .828	.853 / .928	.945 / .923

- Adjacency rows with edge bias: $m^* = 5.6, 5.7, 4.1, 5.9$ at $n = 32, 40, 48, 56$ (seed means). The fitted slope is about 0: over this range the width needed does not grow with n. $m = 2$, one 2-wide head, stays at always-yes for every n; the step from failing to about 0.98 lies between $m = 4$ and 6.
- At $m = 8$ head dimension 2 (4 heads) and 8 (1 head) give the same accuracy (e.g. $n = 56$: 0.971 / 0.975 against 0.974 / 0.867), so the earlier "≤ 8" entries stand.
- Random IDs: $m^* = 4.6, 7.8, > 8, > 8$. Accuracy is not monotone in m ($n = 40$: 0.986 at $m = 6$, 0.931 at $m = 8$), the same instability as for random IDs elsewhere.

6.5. Larger graphs: n = 72–128

In plain words: does the needed width stay small for bigger graphs?

`eb_adj_big`: adjacency rows, bias start 2, head dimension 8, q3lr protocol. Final test pair accuracy (train), seed 0 / seed 1. A dagger marks runs whose best validation epoch was the first, i.e. exact-match accuracy never left its always-yes value.

m	$n = 72$	$n = 96$	$n = 128$
4	.780 / .801	.808† / .788†	–
8	.896 / .909	.831† / .799†	–
12	.935 / .851	.827† / .828†	–
16	.987 / .983	.925† / .959	.890† / .855†
24	–	.930 / .948	.916† / .964
32	–	.968 / .968	.930 / .929†

- m^*, seed 0 / seed 1: $n = 72$ 13.0 / 14.9; $n = 96$ 28.0 / 15.7; $n = 128 > 32$ / 22.8. Train equals test throughout, so data is not the limit.
- Accuracy is not monotone in m at $n = 128$: seed 1 passes at $m = 24$ and fails at $m = 32$. The training curves show why. Every run first sits at always-yes (pair 0.77) for 4–7 epochs, then climbs to a second plateau at 0.85–0.93. Some runs leave it before the cosine schedule reaches zero, some do not, and that, not the width, sets m^*.

More, narrower heads do not change this. `eb_adj_hd2_72` / `_96` repeat the head dimension 2 setting that kept m^* near 6 up to $n = 56$. Final test pair accuracy, seed 0 / seed 1:

n	$m = 4$	6	8	12	16
72	.916 / .900	.892 / .928	.933 / .949	.952 / .946	–
96	–	.856 / .849	.926 / .865	.926 / .911	.932 / .959

m^* is 11.5 / > 12 at $n = 72$ and > 16 / 15.2 at $n = 96$, the same rise as with head dimension 8. The rise past $n = 56$ does not come from too few heads.

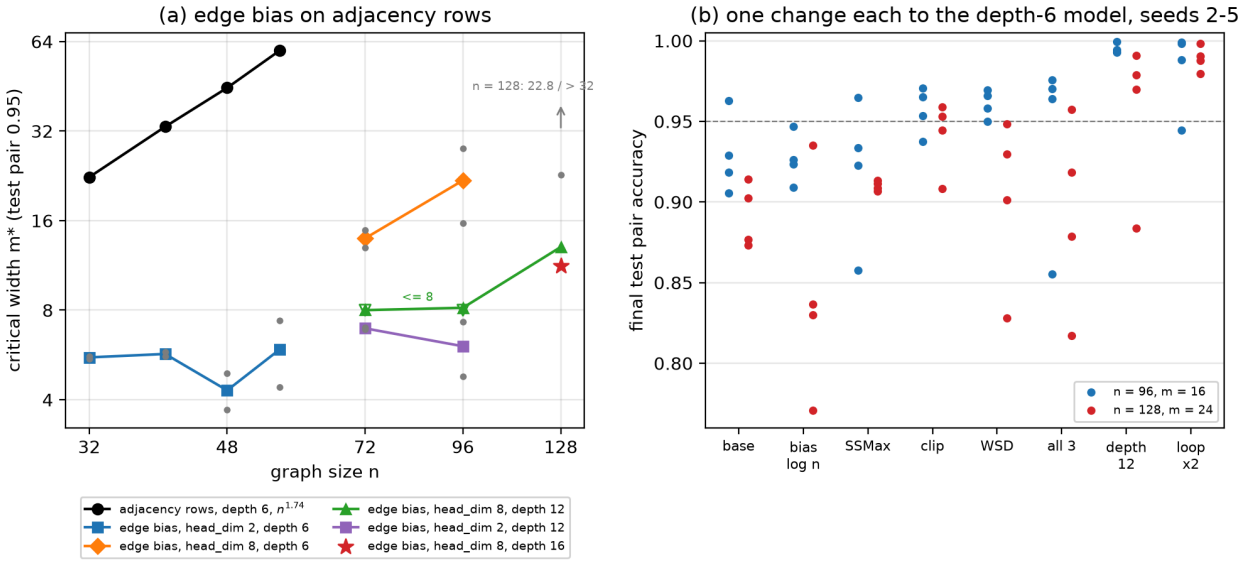


Figure 2: (a) m^* against n for edge bias on adjacency rows. Depth 6: head dimension 2 for $n \leq 56$, 8 for $n \geq 72$; grey dots are per-seed crossings. Green: depth 12 (Section 8), open triangles where the smallest width tried already passes; purple: depth 12 with head dimension 2; red star: depth 16 at $n = 128$. (b) The plateau ablation of Section 7.4: final test pair accuracy per seed, one change per column, blue $n = 96$, red $n = 128$.

7. Why edge-bias models stall at n ≥ 96

In plain words: from 96 nodes on, many edge-bias models stop improving partway: they get 85–93 % of pairs right and stay there. Making them wider did not reliably help. This section finds out why, first by inspecting the trained models and then by changing one thing at a time.

7.1. Hypotheses

A Claude Research report on the plateau (2026-11-05) proposed five possible causes:

- **H1, the bonus fades with size.** The attention bonus b for neighbours is one fixed number, but a node has only about 3.5 neighbours and n other nodes competing for attention. At the start of training the neighbours get about 48 % of the attention at $n = 32$ and only 17 % at $n = 128$, so the "follow the edges" signal is weaker in big graphs.
- **H2, luck.** Getting off a plateau is a sudden, partly random event, so some seeds escape and some do not.
- **H3, the learning rate runs out.** The learning rate is lowered to 0 by the end of training, so a run that is about to escape late may not have enough left.
- **H4, unbalanced answers.** Most pairs are "yes", so the loss may not push hard on the "no" pairs.
- **H5, not enough layers.** Each layer moves information about one step, so six layers reach about 12 steps; the report claimed the graphs outgrow that between $n = 72$ and 96.

7.2. Checks without a GPU

Hop distances in the data (400 graphs per size, pairs in the same component of connected graphs):

n	median	90th percentile	pairs > 12 hops
72	6	10	0.6 %
96	7	11	3.4 %
128	7	12	9.0 %

Distances grow smoothly, with no jump between $n = 72$ and 96, and pairs beyond 12 hops are fewer than the 7–15 % of pairs the stalled models get wrong. A hard 12-hop wall cannot be the whole cause.

`eb_diagnose.py` on all 22 saved `eb_adj_big` models:

- The model does raise b, from 2 to about 3.3–4 in layers 1–2 at every n, but that still puts only 31–49 % of the attention on neighbours at $n = 128$ against 55–63 % at $n = 72$. Dilution is real, and the model does not undo it.
- Middle layers often lower b to about 0 or below (content-based or global attention), in stalled and escaped runs alike.
- Pair error rises with hop distance in every model. Stalled models are 2–3× worse at every distance, short ones included. The clearest marker of a stalled run is the error on pairs in different components: 12–32 % against 6 % for escaped runs.

7.3. Checkpoint surgery

In plain words: take a model that is stuck, change one thing, and train a little more. If the change unsticks it, that thing was the obstacle.

`eb_surgery.py` resumes four stalled checkpoints for 6000 steps (a quarter of the original budget, fresh Adam) with one change each. Final test pair accuracy; the value right after the change in brackets:

checkpoint (start)	as is, LR 10^{-4}	LR 10^{-3}	$b + 1.5$	+ 6 identity blocks	blocks run twice
$n = 128$, $m = 16$, s1 (.855)	.857	.857	.855 (.693)	.980	**.994** (.703)
$n = 128$, $m = 32$, s0 (.930)	.940	.946	.942 (.858)	.983	**1.000** (.859)
$n = 128$, $m = 32$, s1 (.929)	.932	.951	.932 (.825)	.995	**1.000** (.906)
$n = 96$, $m = 16$, s0 (.925)	.930	.931	.933 (.835)	.995	**.998** (.773)

More training, a re-warmed learning rate and a stronger edge bias leave every model on the plateau. Extra depth frees every one of them: six new blocks that start as the identity reach 0.980–0.995, and running the model's own six blocks twice, with no new parameters at all, reaches 0.994–1.000.

7.4. Ablation from scratch

In plain words: the same question from the other side: train new models with one change each and count how many of 4 seeds reach 0.95.

`ebx96_*`: $n = 96$, $m = 16$, full q3lr budget, four new seeds (2–5), one change each (Figure 2b). Final test pair accuracy:

change	seed 2	3	4	5	≥ 0.95
none	.905	.918	.929	.963	1 / 4
bias start $2 + \ln((n - 3.5)/28.5)$	.909	.923	.926	.947	0 / 4
bias × $\ln n / \ln 32$	.857	.934	.965	.923	1 / 4
gradient clipping at 1	.965	.937	.953	.971	3 / 4
WSD schedule	.958	.950	.966	.969	3 / 4
the three above	.855	.976	.964	.970	3 / 4
depth 12	**.994**	**.993**	**.999**	**.993**	**4 / 4**
6 blocks run twice	.999	.988	.998	.945	3 / 4

- The two fixes for dilution (H1) do not help. Clipping and WSD (H3) help a little, but the runs they rescue sit just above 0.95.
- Depth decides it: depth 12 passes in every seed at about 0.99 or more, and weight-tied looping does almost as well with the parameters of the depth-6 model.

The same ablation at $n = 128$, $m = 24$ (`ebx128_*`, Kaggle, red in Figure 2b):

change	seed 2	3	4	5	≥ 0.95
none	.877	.902	.914	.873	0 / 4
bias start $2 + \ln((n - 3.5)/28.5)$	.771	.837	.830	.935	0 / 4
bias $\times \ln n / \ln 32$	.913	.907	.909	.911	0 / 4
gradient clipping at 1	.959	.953	.945	.908	2 / 4
WSD schedule	.828	.930	.948	.901	0 / 4
the three above	.817	.918	.879	.958	1 / 4
depth 12	.884	.979	.970	.991	3 / 4
6 blocks run twice	**.990**	**.988**	**.980**	**.998**	**4 / 4**

- The unchanged model never escapes at $n = 128$ (0 / 4, against 1 / 4 at $n = 96$). The bias fixes and WSD do nothing; clipping rescues half, again just above 0.95.
- Depth again decides it. Running the six blocks twice is the most reliable change here (0.98–1.00 in every seed); one depth-12 seed stays at 0.884.

7.5. Conclusion

In plain words: each layer moves information about one step along the edges. Past about 96 nodes, six layers are not enough to carry it reliably across a component, and adding width cannot replace the missing layers. More layers, or reusing the same layers twice, fixes it.

The stall is a depth limit, not a width or schedule limit. With the edges in the attention, each layer passes information about one hop, and at $n \geq 96$ six layers do so too weakly (errors at every distance, not only beyond 12 hops). The rise of m^* past $n \approx 56$ in Figure 2a therefore measures depth: at a fixed depth of 6 the width compensates for the missing depth, badly and with seed-dependent results. For this encoding the width study has to hold depth in a fixed relation to n (for example $\lceil \log_2 n \rceil$) or report $m^*(n, L)$ on a small depth grid. That matches the theory the project builds on, where connectivity needs depth $\Theta(\log n)$ or width about n.

8. Width at depth 12

In plain words: if six layers were the obstacle, give every model twelve and measure the width again. If the width curve goes flat, the earlier rise was all depth.

`ebd12_72` / `_96` / `_128`: the `eb_adj_big` setting (adjacency rows, bias start 2, head dimension 8, q3lr budget) at depth 12, $m = 8$, 12, 16, 24, seeds 0 / 1. Final test pair accuracy:

n	seed	$m = 8$	12	16	24	m^*, depth 12	m^*, depth 6
72	0	.990	.960	.997	.999	≤ 8	13.0
72	1	.995	.995	.986	.994	≤ 8	14.9
96	0	.949	.959	.926	.993	8.3	28.0
96	1	.988	.815	.990	.992	≤ 8	15.7
128	0	.869	.868	.976	.941	14.9	> 32
128	1	.946	.921	.966	.994	14.4	22.8

- **Depth was most of the rise.** At $n = 72$ and 96 the smallest width tried already passes in 3 of 4 cases; at $n = 128$ m^* falls from 23 / > 32 to about 14.5.

- **Not flat yet.** $n = 128$ needs about twice the width of $n = 96$. Either twelve layers are again short at $n = 128$ (the ablation's one failed depth-12 seed points that way), or the narrow models are limited by having a single 8-wide head at $m = 8$ and 12.
- **Single runs are erratic.** $n = 96$ seed 1 passes at $m = 8$ and 16 but reaches only 0.815 at 12; $n = 96$, $m = 16$, seed 0 reached 0.987 at epoch 14 and ended at 0.926; at $n = 128$ seed 0 is worse at 24 than at 16. One run per seed and width gives a noisy first crossing.
- **Not deterministic.** A sharding bug ran $n = 72$, $m = 8$, seed 1 twice on the same GPU type: 0.995 and 0.938. Whether a run leaves the plateau is partly chance, so the readout should be the fraction of seeds passing at each width, not one crossing per seed.

8.1. Below width 8: head dimension 2 at depth 12

In plain words: with 12 layers the smallest width tried (8) already passed at 72 and 96 nodes. Two numbers per head allow widths 4 and 6, as in Section 6.4.

`ebd12hd2_72` / `_96`: as above, head dimension 2, $m = 4$, 6, 8. Final test pair accuracy:

n	seed	$m = 4$	6	8	m^*
72	0	.764	.893	.999	7.0
72	1	.764	.905	1.000	6.9
96	0	.915	.996	.975	4.8
96	1	.768	.862	.994	7.3

m^* is about 7 at $n = 72$ and 96, against 4.1–5.9 at $n = 32$–56 with six layers (purple in Figure 2a). Once depth is not the limit, the width this encoding needs barely grows from 32 to 96 nodes.

8.2. Is twelve layers short at n = 128?

In plain words: at 128 nodes the twelve-layer models still needed about twice the width. If twelve layers are again too few there, sixteen should bring the width back down.

`ebd16_128`: depth 16, seeds 0–3; `ebd12_128x` adds seeds 2–3 at depth 12 so both depths have four seeds at $m = 8$, 12, 16 (head dimension 8). Final test pair accuracy, the maximum during training in brackets where it was more than 0.01 higher:

depth	m	seed 0	1	2	3	≥ 0.95
12	8	.869	.946	.958	.876	1 / 4
12	12	.868	.921	.927	.908	0 / 4
12	16	.976	.966	.965	.964	**4 / 4**
16	8	.840	.869	.994	.848	1 / 4
16	12	.972	.771	.831	.983	2 / 4
16	16	.999	.999	.869 (.938)	.901 (.985)	2 / 4

- m^* per seed: depth 12 14.9 / 14.4 / ≤ 8 / 14.9; depth 16 11.2 / 15.0 / ≤ 8 / 10.9. Sixteen layers help a little at $m = 12$, not at $m = 16$; neither depth brings $n = 128$ down to the ≈ 7 of $n = 72$ / 96 (red star in Figure 2a).
- The deeper models are less stable. Two depth-16 runs at $m = 16$ reached 0.94 and 0.985 and then fell back. Late collapses now cost as much as the plateau did at depth 6.
- So at $n = 128$ either the width really grows, or training at this depth is unreliable enough to hide it. Gradient clipping (2 / 4 in the $n = 128$ ablation) is the obvious control before reading this as width growth. Head dimension 8 against 2 is a second difference from the $n = 72$ / 96 points.

9. Operations

- **Lost Colab session (2026-11-03).** The first `q3n72` runtime died about 30 min after the laptop went to sleep on battery with the lid closed; 5 of 8 runs were lost because results lived only on the VM. Fix: a local watcher polls every 5 min with a 120 s timeout on every Colab call, downloads results as they appear, stops the VM when done, and is tied to `caffeinate`; the laptop stays on AC power with the lid open.
- **Missing dependency.** The first `n72c` launch failed: the Colab image lacks `torch_geometric`. The run script now installs it.
- **Memory.** Building 128k graphs at $n = 96$ in float32 peaked at 20 GB per process; data are now built as uint8. A 256k set at $n = 72$ (5 GB peak while built) needs a high-RAM VM.
- **Lost Colab session (2026-11-05).** The first n = 96 / 128 VM died about two hours in, at the moment the laptop lost its network connection (a DNS failure in the watcher log); 4 of 10 runs survived. The Colab runtime ends when the local client drops off, so the laptop must stay online, not only awake; the watcher now runs under `caffeinate -ims`.
- **GPU memory at n = 128.** Four workers on one T4 ran out of memory loading the n = 128 data (about 4 GB per worker on the GPU). Colab now runs 2 workers; the Kaggle notebook gained a job queue with at most 4 workers alive, 2 per GPU.
- **Kaggle hang (2026-11-06).** The first `ebx128` session printed nothing after about 35 min and was cut off at 12 h with no results. Building four $n = 128$ datasets at once (about 8 GB each while built) exceeded Kaggle's 30 GB; one builder was killed and Python's process pool waited for it forever. `--prepare` now limits parallel builds by RAM (2 at $n = 128$).
- **Shard bug (2026-11-06).** Shards split the runs still pending, so a shard started after another had saved a run took a shifted slice: in `ebd12`, 3 runs ran twice and 6 never. Shards now take a fixed slice of the full list; the 6 runs were filled in on a second VM.
- **Compute.** Colab Pro: 91.45 → 47.71 units over ten sessions (T4 ≈ 1.15 units/h, high RAM ≈ 1.6). Kaggle: each sweep within one session.

10. Open items

1. $n = 128$ at depth 12–16 with gradient clipping and head dimension 2, 4 seeds, pass fraction as the readout: is the rise to $m^* \approx 11$–15 real width growth or unstable training?
2. Weight-tied looping as the cheaper way to add depth: it matched or beat depth 12 in both ablations with no extra parameters.
3. $n = 72$ protocol: 256k graphs with more steps, $m = 96$–192, to get a test m^*.
4. Random IDs: test depth 12 / looping there too; the instability may be the same depth limit.
5. Thesis chapter 3: add the auxiliary-loss result; update the old interval 1.59–1.90 in the Явц 2 slides and `width-progress.typ` to 1.53–1.97.